



Universidad
Virtual

Desarrollo Móvil

PWA - Hora del sistema

Maestría en Ingeniería y Desarrollo de Software

Mtro. Francisco Javier Rivera Magallanes

Presentado por: Elizabeth Orozco Torres

ÍNDICE

1. INTRODUCCIÓN.....	3
2. DESCRIPCIÓN DEL PROYECTO	4
3. DESARROLLO DE LA PWA.....	5
4. IMPLEMENTACIÓN Y REGISTRO DEL SERVICE WORKER.....	6
5. ELABORACIÓN Y CONFIGURACIÓN DEL ARCHIVO DE MANIFIESTO	9
6. REPOSITORIO DEL PROYECTO EN GITHUB	11
7. GUIA PRÁCTICA PARA INSTALACIÓN.....	12
8. CONCLUSIONES	27
9. BIBLIOGRAFÍA.....	28

1. INTRODUCCIÓN

En la actualidad, la línea que divide a las aplicaciones de escritorio, los dispositivos móviles y los navegadores web es cada vez más delgada. El usuario moderno ya no busca solo información, sino experiencias que sean inmediatas, accesibles y que funcionen en cualquier lugar, independientemente de su conexión a internet. Como bien señala Frances Berriman, una de las mentes detrás del concepto original de estas tecnologías: *"Las aplicaciones web progresivas no son solo una tecnología, sino un cambio en la forma en que pensamos sobre el software: se trata de eliminar las fricciones entre el usuario y la web"* (Berriman, 2017).

Bajo esta premisa, el presente proyecto tiene como objetivo el desarrollo de una aplicación web de tiempo real diseñada para mostrar la hora del sistema. A través de este proyecto, se busca crear una herramienta que no solo sea informativa, sino que sea capaz de integrarse de forma fluida en el dispositivo del usuario, permitiendo su uso de manera continua y automática.

2. DESCRIPCIÓN DEL PROYECTO

El presente proyecto consiste en el desarrollo de una Progressive Web App (PWA) diseñada para mostrar la hora exacta del sistema con actualización automática cada segundo. El objetivo principal es aplicar los conceptos de desarrollo multiplataforma y las capacidades modernas de las APIs web para crear una herramienta que combine el alcance de una página web con la experiencia inmersiva de una aplicación nativa.

Para lograr una aplicación confiable, rápida y cautivadora, el desarrollo se apoya en tres pilares fundamentales extraídos del programa de estudio:

1. **Service Worker:** Un intermediario que se ejecuta en segundo plano para gestionar eventos de red (fetch) y permitir el funcionamiento offline.
2. **Web App Manifest:** Un archivo JSON que define la identidad visual (iconos, colores y orientación) y permite que la app sea instalable en dispositivos móviles.
3. **Lógica Asíncrona:** Implementación de JavaScript para garantizar que la interfaz se mantenga actualizada sin intervención del usuario.

3. DESARROLLO DE LA PWA

El primer objetivo del proyecto consistió en el desarrollo de una Progressive Web App (PWA) diseñada para la visualización precisa y constante de la hora del sistema. A diferencia de una página web tradicional, esta aplicación aprovecha las capacidades de los navegadores modernos para funcionar como un software autónomo e instalable en dispositivos móviles y de escritorio.

La funcionalidad principal se basa en una lógica de actualización asíncrona, eliminando cualquier necesidad de intervención por parte del usuario o recargas manuales del sitio. Para lograrlo, se implementó un motor en JavaScript que utiliza el método `setInterval`, configurado para ejecutarse cada 1000 milisegundos. En cada ciclo, la aplicación captura el objeto `Date` del sistema, extrae la hora local y la inyecta directamente en la interfaz.

Este proceso garantiza que el reloj mantenga una fluidez absoluta, comportándose con la misma eficiencia que una aplicación nativa. Además, gracias a su arquitectura como PWA, esta actualización ocurre de manera ligera, consumiendo el mínimo de recursos y manteniendo la precisión del sistema operativo en todo momento.

4. IMPLEMENTACIÓN Y REGISTRO DEL SERVICE WORKER

El núcleo de la "inteligencia" de esta PWA reside en su Service Worker, un archivo JavaScript que actúa como un proxy de red ejecutándose en un hilo separado de la página principal. Siguiendo los criterios de una aplicación web progresiva, este componente se diseñó para cumplir con dos funciones críticas: el funcionamiento offline y la optimización del rendimiento.

A. Implementación Técnica (sw.js):

El Service Worker fue programado para gestionar el ciclo de vida de la aplicación mediante dos eventos principales:

- a) **Instalación (install):** Durante esta fase, se creó un almacén de caché (denominado reloj-pwa-v8) donde se guardaron de forma preventiva todos los recursos esenciales: el archivo HTML, los estilos CSS, la lógica del reloj y los iconos del manifiesto. Esto garantiza que la aplicación sea "Confiable", permitiendo que cargue instantáneamente incluso sin conexión a internet.
- b) **Intercepción de Peticiones (fetch):** Se implementó una estrategia de "Cache First". Cada vez que el usuario intenta acceder a la app, el Service Worker intercepta la petición y entrega los archivos directamente desde la memoria local, reduciendo el tráfico de datos y acelerando la velocidad de respuesta.

B. Registro de la Aplicación (app.js):

Para que el navegador active estas funciones, se incluyó un bloque de código en el script principal que verifica la compatibilidad del sistema. Una vez confirmada, se procede al registro formal del archivo sw.js. Se añadió además un mecanismo de actualización automática mediante un sello de tiempo (Date.now()), asegurando que cualquier mejora en el diseño o la lógica se despliegue de forma inmediata en el dispositivo del usuario sin intervención manual.

Código de implementación y registro

Implementación Técnica (sw.js)

Este archivo gestiona el almacenamiento en caché para permitir el uso sin conexión a internet.

```
// Nombre
const NOMBRE_CACHE = 'reloj-pwa-v8';

// Archivos que funcionan offline
const ARCHIVOS_PARA_CACHE = [
  './',
  './index.html',
  './style.css',
  './app.js',
  './manifest.json',
  './icon-192.png',
  './icon-512.png'
];

// Evento de INSTALACIÓN: Se ejecuta la primera vez que se abre la app
self.addEventListener('install', evento => {
  evento.waitUntil(
    caches.open(NOMBRE_CACHE)
      .then(cache => {
        console.log('Service Worker: Guardando archivos en caché...');
        return cache.addAll(ARCHIVOS_PARA_CACHE);
      })
  );
});

// Evento FETCH: Se ejecuta cada vez que la app pide un archivo al servidor
self.addEventListener('fetch', evento => {
  evento.respondWith(
    caches.match(evento.request)
      .then(respuestaCaché => {
        // Si el archivo está en la caché, lo entrega. Si no, lo busca en internet.
        return respuestaCaché || fetch(evento.request);
      })
  );
});
```

Registro en la Aplicación (app.js)

Este bloque de código, ubicado al final del script principal, activa el Service Worker y gestiona su actualización.

```
// Función principal para obtener y mostrar la hora del sistema
function actualizarHoraSistema() {
  const elementoReloj = document.getElementById('reloj');
  const ahora = new Date();

  // Convertimos la hora a un formato legible (HH:MM:SS)
  const horaFormateada = ahora.toLocaleTimeString();

  // Insertamos la hora en el HTML
  elementoReloj.textContent = horaFormateada;
}

// Configuramos el temporizador para que ejecute la función cada 1000 milisegundos (1 segundo)
setInterval(actualizarHoraSistema, 1000);

// Llamamos a la función una vez al inicio para que no aparezca en 00:00:00 el primer segundo
actualizarHoraSistema();

// REGISTRO DEL SERVICE WORKER
// Verificamos si el navegador es compatible con Service Workers
if ('serviceWorker' in navigator) {
  window.addEventListener('load', () => {
    navigator.serviceWorker.register('./sw.js')
      .then(registro => {
        console.log('Service Worker registrado con éxito en:', registro.scope);
      })
      .catch(error => {
        console.error('Fallo al registrar el Service Worker:', error);
      });
  });
}
```


5. ELABORACIÓN Y CONFIGURACIÓN DEL ARCHIVO DE MANIFIESTO

El archivo de manifiesto es un componente en formato JSON cuya función principal es controlar la apariencia de la aplicación ante el usuario final. Es el encargado de indicarle al sistema operativo (Android, iOS o Windows) que el sitio web debe comportarse como una aplicación nativa, permitiendo su instalación en la pantalla de inicio sin necesidad de pasar por una tienda de descargas tradicional.

Siguiendo los requerimientos de la consigna, se configuraron los siguientes parámetros técnicos para garantizar una experiencia de usuario fluida y coherente:

- 🚦 **Identidad Visual:** Se definió el nombre completo (Reloj Digital PWA) y un nombre corto para el menú del dispositivo.
- 🚦 **Modo de Visualización:** Se utilizó la configuración standalone, la cual elimina las barras de navegación del explorador, brindando una sensación de inmersión total.
- 🚦 **Personalización de Colores:** Se estableció un `background_color` para la pantalla de carga (Splash Screen) y un `theme_color` que tiñe la barra de estado del móvil, logrando una estética profesional.
- 🚦 **Adaptabilidad:** Se fijó la orientación en modo retrato (portrait) para asegurar que el diseño del despertador se visualice siempre de forma óptima.
- 🚦 **Iconografía:** Se vincularon iconos de 192px y 512px, creados a partir de capturas reales de la interfaz, para asegurar nitidez en cualquier resolución de pantalla.

Código del Manifiesto Implementado (**manifest.json**)

```
{
  "name": "Reloj Digital PWA",
  "short_name": "RelojPWA",
  "description": "Una aplicación simple que muestra la hora exacta y funciona offline.",
  "start_url": "./index.html",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#2c3e50",
  "orientation": "portrait",
  "icons": [
    {
      "src": "icon-192.png",
      "sizes": "192x192",
      "type": "image/png",
      "purpose": "any"
    },
    {
      "src": "icon-512.png",
      "sizes": "512x512",
      "type": "image/png",
      "purpose": "any"
    }
  ]
}
```

6. REPOSITORIO DEL PROYECTO EN GITHUB

La estructura del repositorio fue organizada para cumplir con los estándares técnicos de despliegue y los requerimientos de la consigna. Los archivos se cargaron directamente en la raíz (root) para facilitar su detección por parte del navegador y del Service Worker

Elementos del repositorio:

1. **index.html (Archivo principal):** Contiene la estructura semántica de la aplicación, los metadatos necesarios y los enlaces al manifiesto, la hoja de estilos y el script de lógica.
2. **style.css (Hoja de estilos):** Define la apariencia visual de la interfaz, incluyendo el diseño responsivo del "despertador" digital.
3. **app.js (JavaScript de lógica):** Implementa la función de actualización de la hora cada segundo y el código de registro del Service Worker.
4. **icon-192.png e icon-512.png (Iconos de la app):** Imágenes en formato PNG que sirven como identidad visual para la instalación en dispositivos.
5. **manifest.json (Manifiesto de la PWA):** Archivo de configuración que permite que la web sea reconocida e instalada como una aplicación.
6. **sw.js (Service Worker):** Script encargado de la gestión de caché y el funcionamiento offline del proyecto.

Link del repositorio en GitHub:

<https://github.com/rex2025rex/PWA/blob/main/README.md>

Acceso a la Aplicación:

<https://rex2025rex.github.io/PWA/>

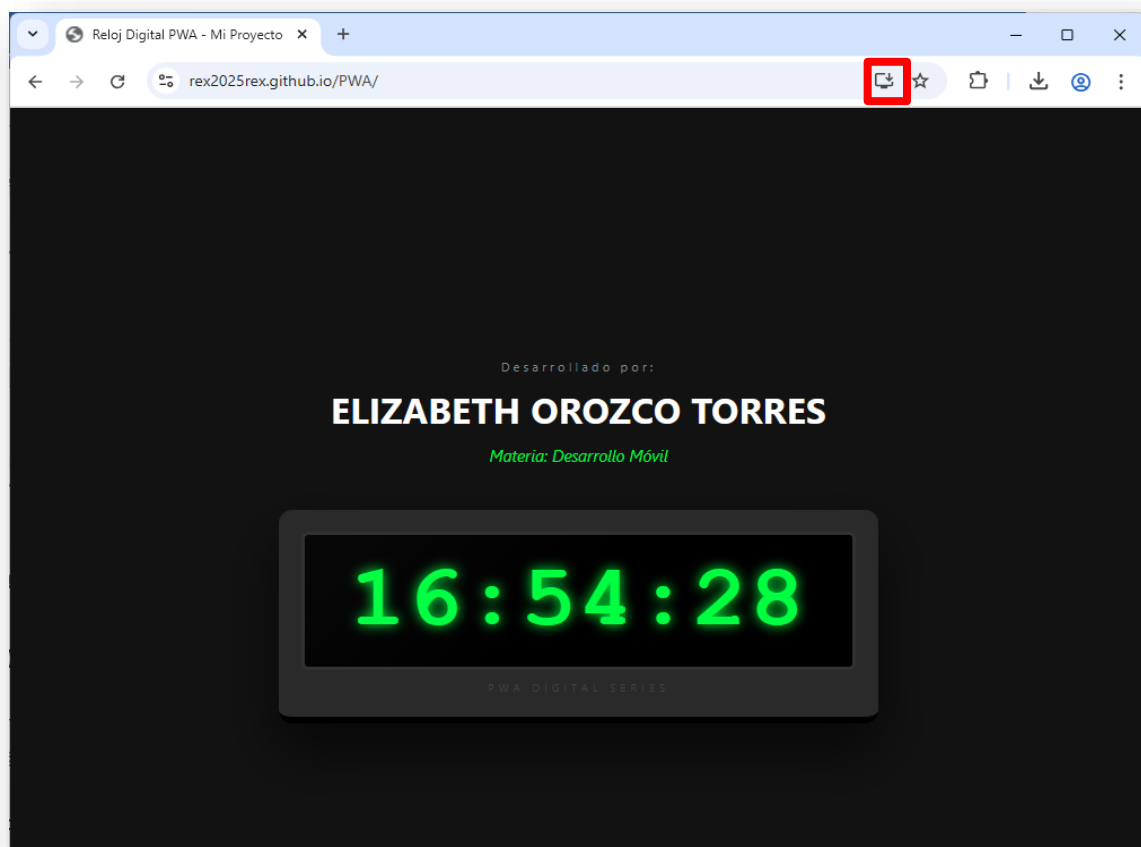
7. GUIA PRÁCTICA PARA INSTALACIÓN

Escritorio (Windows)

Ingresa desde el navegador en **Google Chrome** o **Edge**. al Link:

<https://rex2025rex.github.io/PWA/>

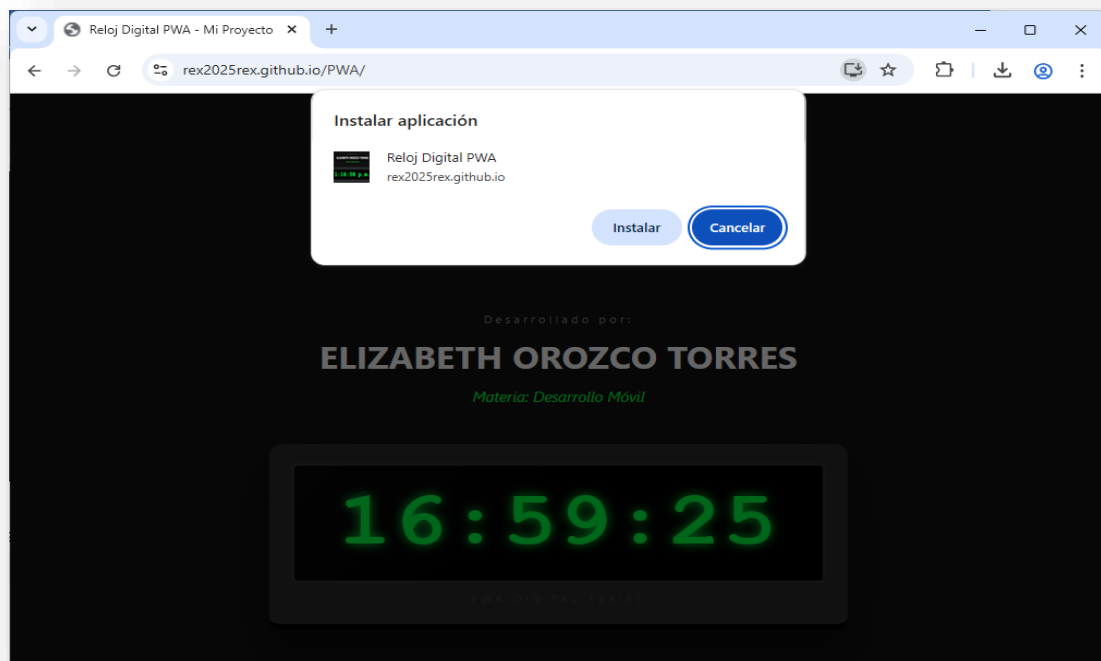
A continuación, se desplegará la siguiente pantalla:



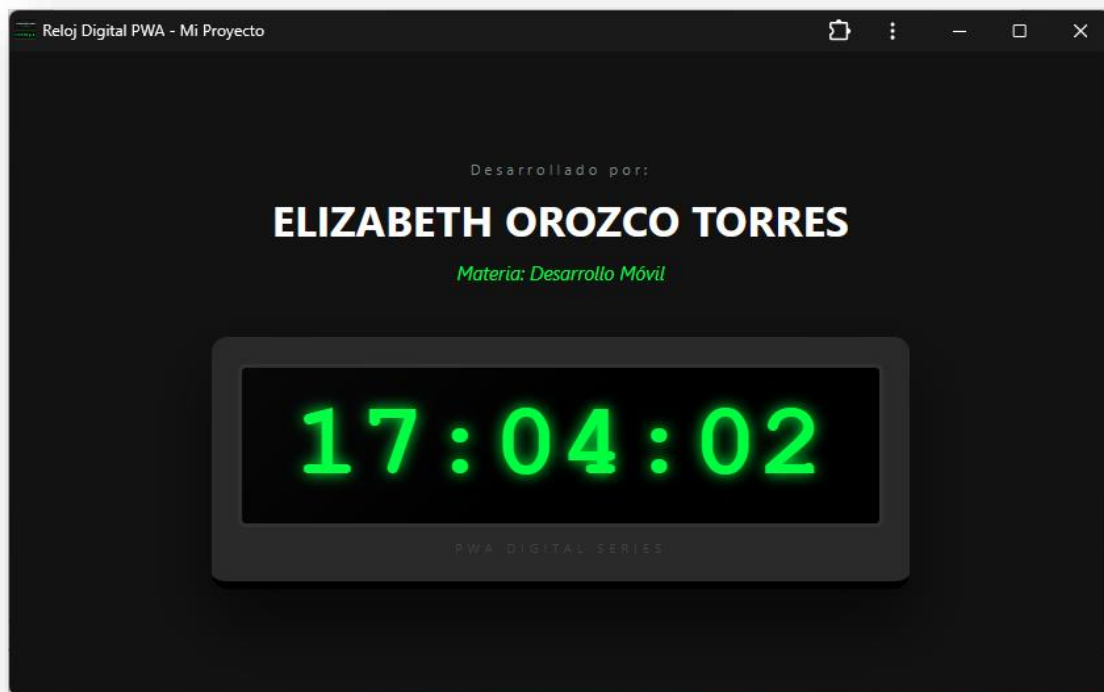
Busca en la barra de direcciones (extremo superior derecho) el icono de una pantalla con una flecha.



Para instalar da click en el icono, se abre la ventana que se muestra a continuación, y nuevamente selecciona instalar:



Concluida la instalación se abrirá la siguiente ventana:

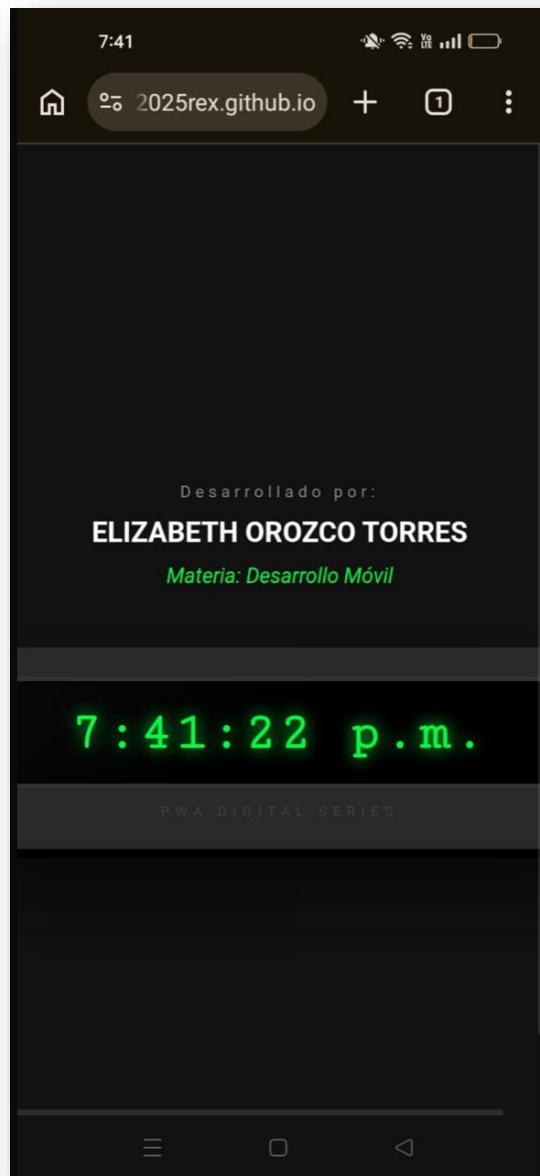


Para comprobar que se instaló correctamente, busque el icono que se agrega en el escritorio como se muestra a continuación:

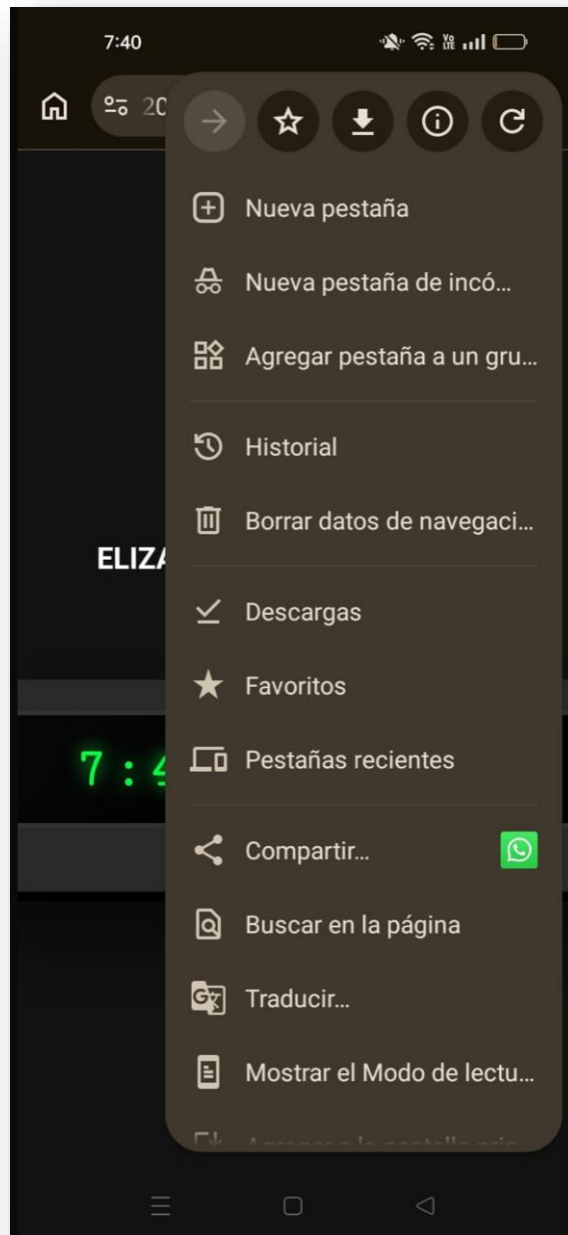


Android (Google Chrome)

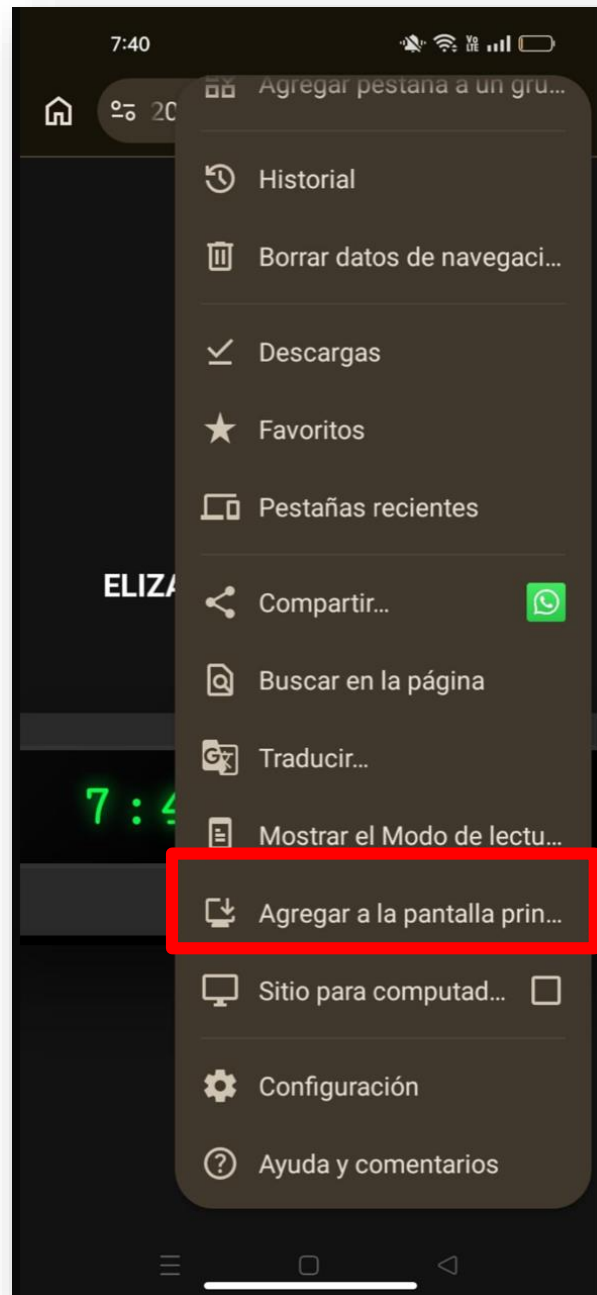
1. **Abrir:** Accede al enlace <https://rex2025rex.github.io/PWA/> desde tu dispositivo.



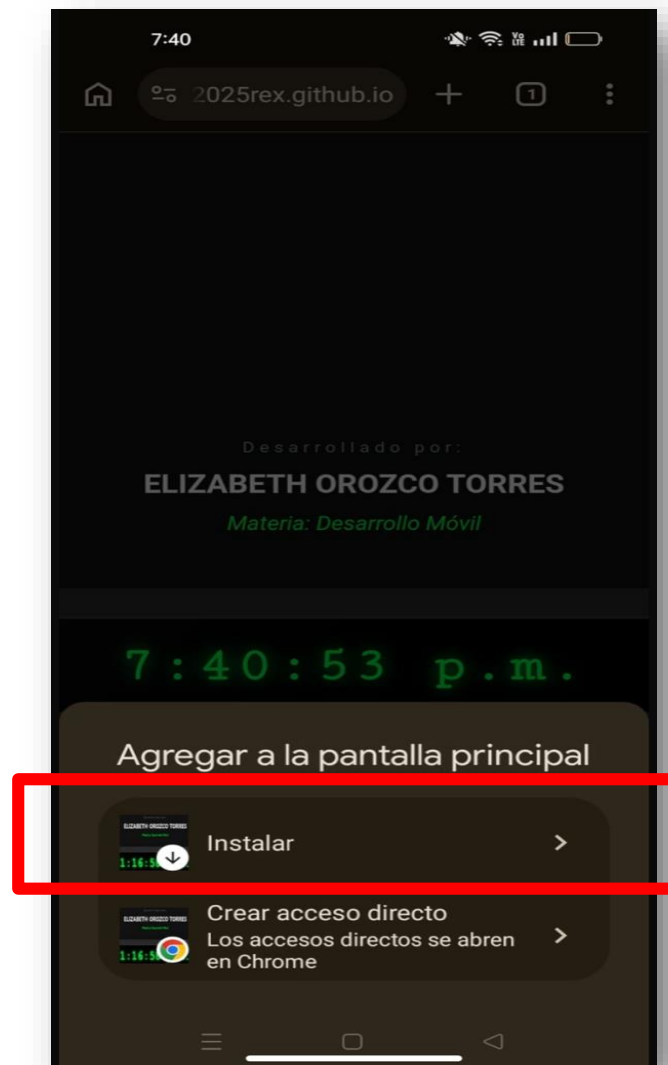
2. **Opciones:** Toca los **tres puntos (:)** en la esquina superior derecha y desliza hacia abajo.



3. **Instalar:** Selecciona la opción **"Agregar a la pantalla de inicio"**.



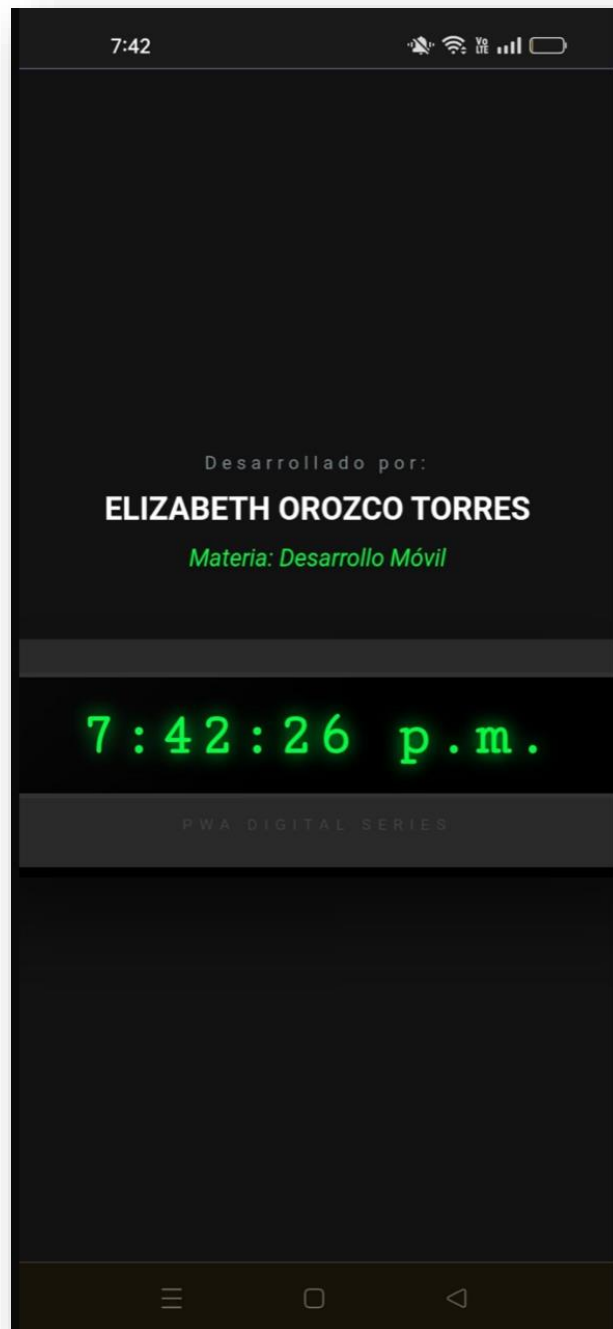
4. **Confirmar:** Presiona "Instalar" y el icono aparecerá junto a tus otras apps.



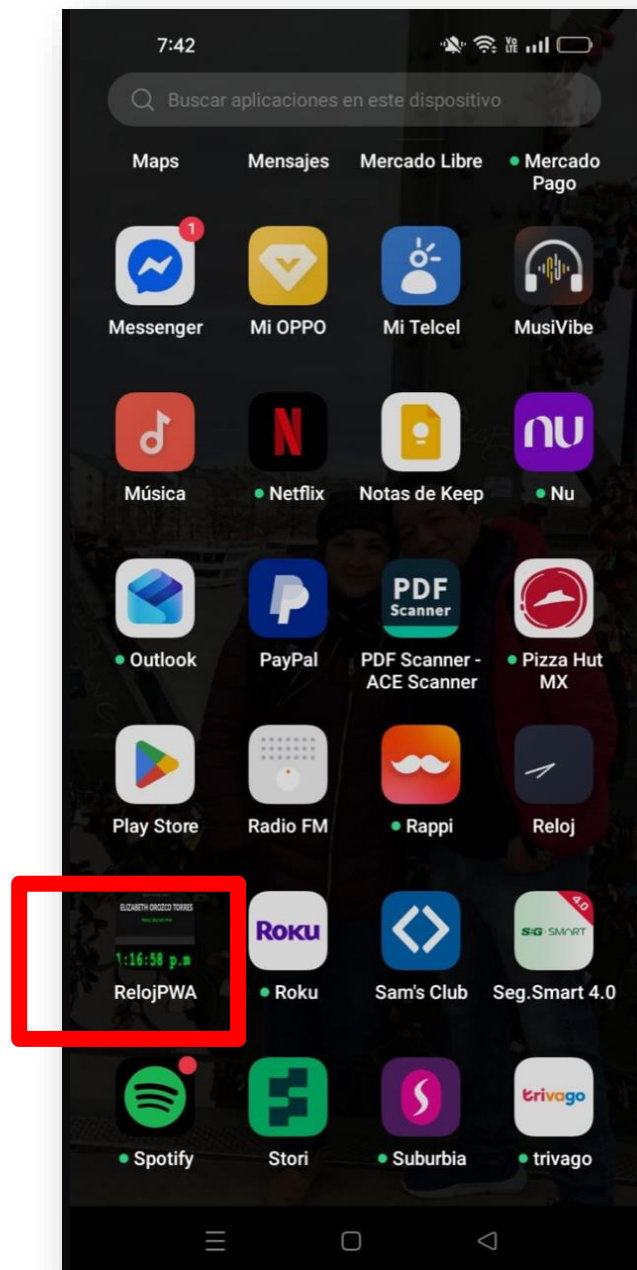
Solicita confirmar la instalación, seleccionar instalar:



5. **Abrir:** Se abre la aplicación



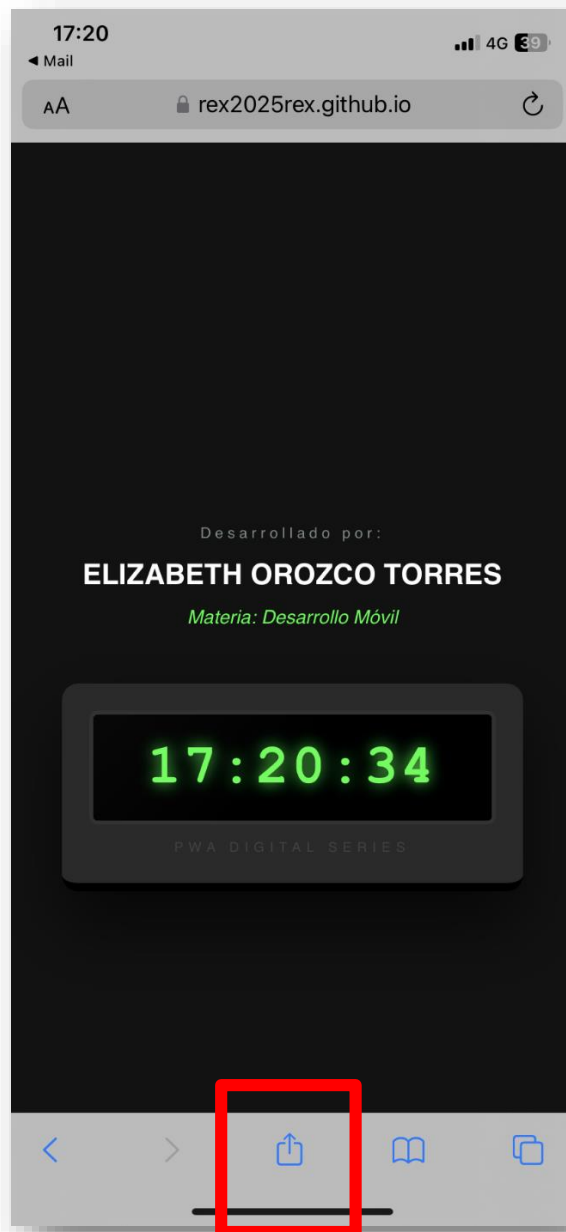
También se crea un icono en el escritorio del móvil.



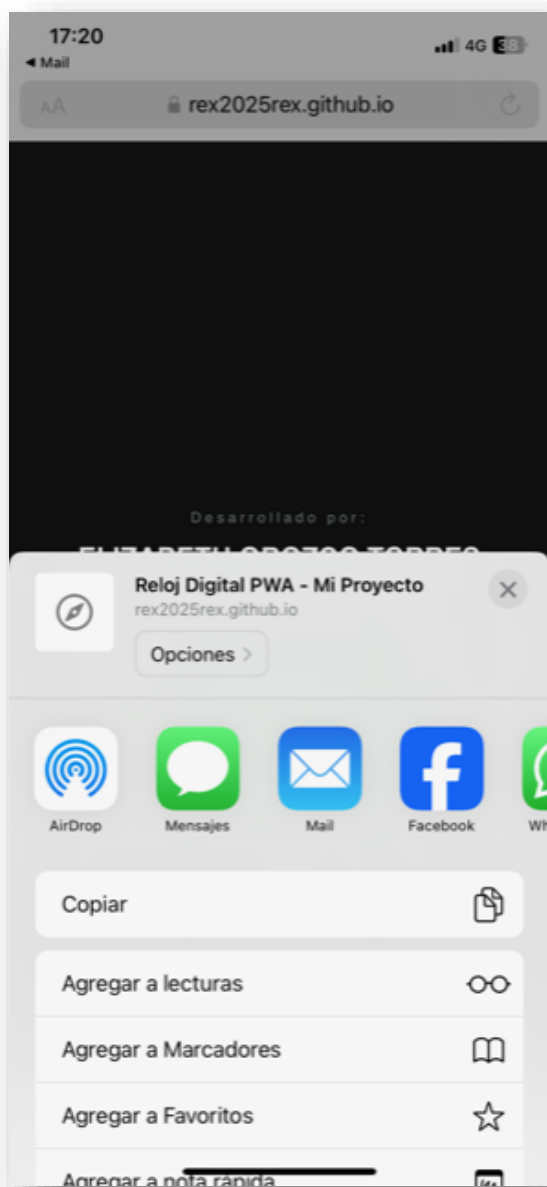
iPhone / iPad (Safari)

Nota: iOS no muestra botones automáticos; debe realizarse manualmente.

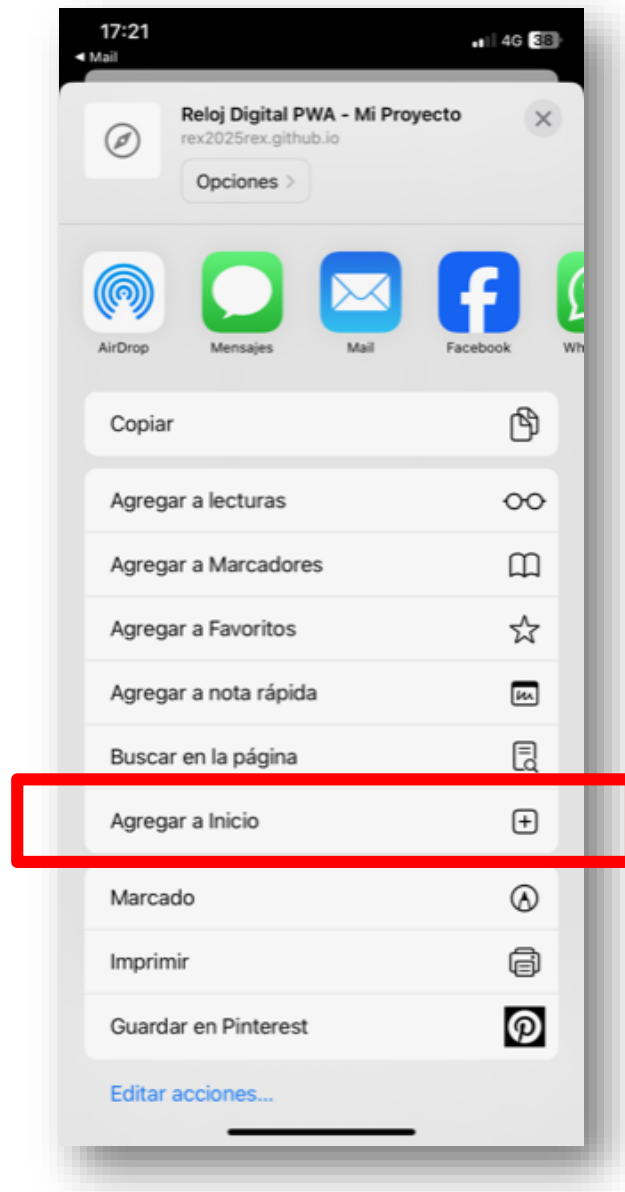
1. **Abrir:** Ingresa al enlace <https://rex2025rex.github.io/PWA/> exclusivamente desde **Safari**.



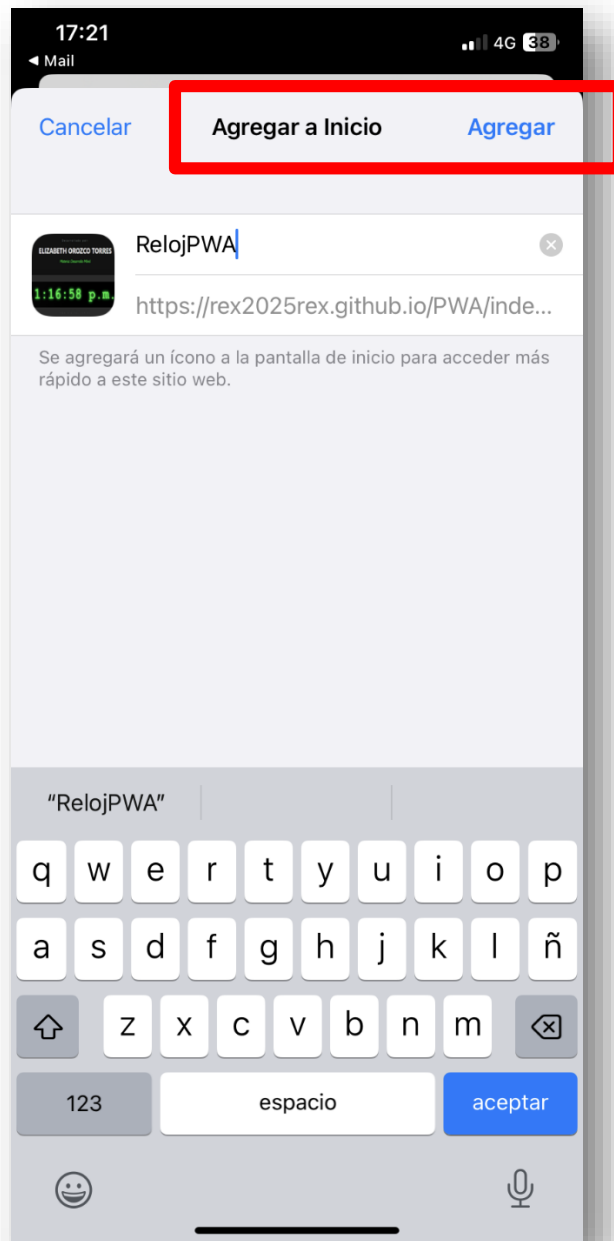
2. **Compartir:** Toca el botón **Compartir** (cuadrado con flecha hacia arriba .



3. **Agregar:** Desliza el menú hacia abajo y selecciona "**Agregar a inicio**".



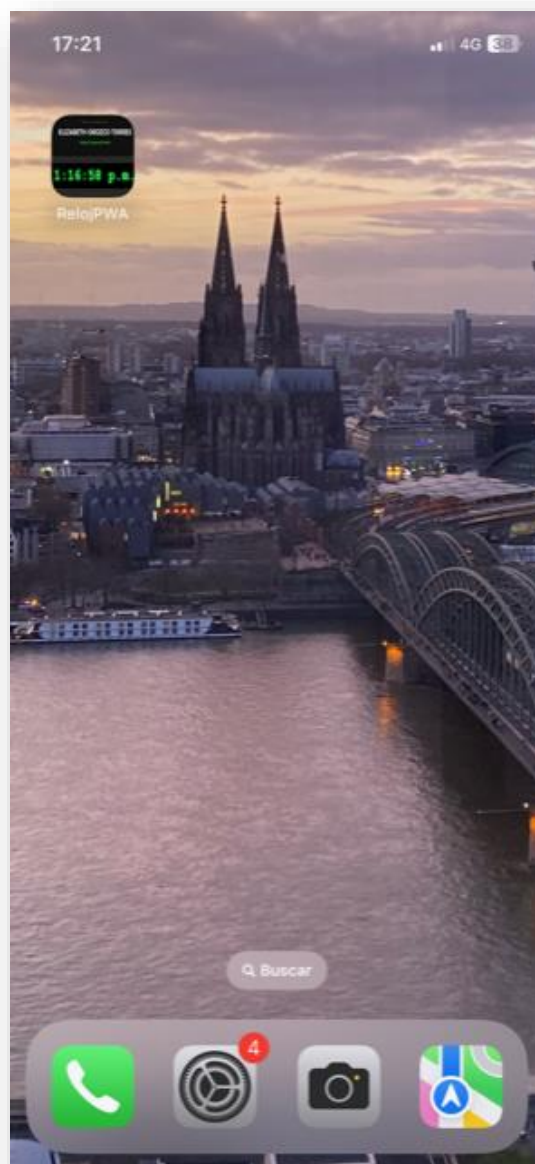
4. **Finalizar:** Toca "**Agregar**" en la esquina superior derecha.



5. **Abrir:** Se abre la aplicación



También se crea un icono en el escritorio del móvil.



8. CONCLUSIONES

El desarrollo de esta Progressive Web App (PWA) demuestra cómo la integración de tecnologías modernas permite transformar una interfaz digital en una herramienta instalable, rápida y eficiente. Al implementar un Service Worker, logramos que la aplicación sea confiable incluso sin conexión a internet, rompiendo la dependencia tradicional de la red y garantizando que el usuario tenga acceso a la información en tiempo real de manera ininterrumpida.

Por otro lado, la configuración del archivo de manifiesto y el diseño responsivo aseguran una experiencia visual inmersiva que se adapta con fluidez a cualquier dispositivo. Esta capacidad multiplataforma, sumada a un despliegue seguro mediante GitHub, no solo moderniza el acceso al software, sino que ofrece una solución profesional y escalable que responde con precisión a las exigencias digitales actuales.

9. BIBLIOGRAFÍA

Berriman, F. (2017). *Progressive Web Apps: The Future of the Web*. fberriman.com

Google Developers. (2024). *Service Workers: An Introduction*. Web Fundamentals. chrome.com

MDN Web Docs. (2024). *Service Worker API*. Mozilla Developer Network. mozilla.org

Microsoft Learn. (2024). *Build Progressive Web Apps (PWA)*. Microsoft Documentation. microsoft.com

World Wide Web Consortium. (2024). *Web App Manifest Specification*. w3.org